

IUT DE NICE COTE D'AZUR - DEPARTEMENT INFORMATIQUE

41 BD. NAPOLEON III - 06200 NICE



Développement d'une application web de centralisation et de suivi des PDF de modes dégradés

Réalisé par Maxime Giovanelli



Centre de Lutte contre le Cancer - Antoine Lacassagne.

33 Av. de Valombrose, 06100 Nice

**Rapport de stage pour la validation de la deuxième année du Bachelor
Universitaire de Technologie Informatique**

Année 2025-2026

Tuteur IUT : M. Olivier Pantz

Tuteur Entreprise : M. Julien Degardin

Remerciements

Je remercie Julien Degardin, responsable au sein de la DSI du Centre Antoine Lacassagne, pour son encadrement pendant ces huit semaines : ses retours techniques et sa formulation claire du besoin métier ont permis de garder un périmètre réaliste.

Je remercie également Olivier Pantz, tuteur pédagogique à l'IUT Nice Côte d'Azur, pour le suivi du stage, ainsi que l'ensemble de l'équipe de la DSI pour son accueil et ses explications sur le fonctionnement informatique de l'établissement.

Résumé

Ce rapport présente un stage de huit semaines réalisé au Centre Antoine Lacassagne, centre de lutte contre le cancer situé à Nice, au sein de la Direction des Systèmes d'Information (DSI).

Quand un outil métier devient indisponible, les équipes soignantes et administratives doivent retrouver rapidement les documents PDF de modes dégradés (formulaires, procédures, fiches réflexes). Ces documents étaient répartis sur des stockages hétérogènes (SFTP, SMB/CIFS, dossier local), sans vue centralisée ni contrôle de fraîcheur pour la DSI.

La problématique du stage était donc la suivante : comment centraliser et fiabiliser l'accès à ces documents critiques, tout en conciliant des exigences qui tirent en sens opposés - garantir la fraîcheur documentaire sans saturer le réseau hospitalier, imposer une sécurité et une traçabilité de niveau HDS sans compliquer l'accès en situation de panne, et rester assez simple pour être exploité durablement par la DSI ?

L'application livrée, Arche (Flask, SQLAlchemy, SQLite, APScheduler, Docker), collecte les PDF depuis trois types de sources, calcule leur fraîcheur, offre une interface web avec viewer PDF, une purge progressive avec corbeille et une API REST. Les opérations sont journalisées ; la sécurité couvre sessions, tokens, CSRF, CSP, chiffrement Fernet et confinement des chemins - exigences issues du contexte HDS.

Le livrable est une base applicative documentée, testée et conteneurisable. Il reste à la valider sur l'infrastructure réelle de la DSI.

Summary

This report presents an eight-week internship at Centre Antoine Lacassagne, a cancer treatment center in Nice, within its IT department (DSI).

When standard tools become unavailable, medical and administrative staff need quick access to degraded-mode PDF documents (forms, procedures, reference sheets). These documents were spread across heterogeneous servers (SFTP, SMB/CIFS, local storage), with no central tool to track freshness or availability.

The main question was: how can access to these critical documents be centralized and secured while reconciling requirements that pull in opposite directions - guaranteeing document freshness without saturating the hospital network, enforcing HDS-level security and traceability without hindering access during outages, and remaining simple enough for the DSI to operate in the long run?

The delivered application, Arche (Flask, SQLAlchemy, SQLite, APScheduler, Docker), collects PDFs from three source types, computes freshness statuses, provides a web interface with an integrated PDF viewer, progressive purge with a trash bin, and a REST API. Operations are logged; security covers sessions, tokens, CSRF, CSP, Fernet encryption and path confinement.

The result is a documented, tested and containerizable application base. The DSI still has to validate it on real infrastructure before production use.

Table des matières

Développement d'une application web de centralisation et de suivi des PDF de modes dégradés.....	1
Remerciements.....	2
Résumé.....	2
Summary.....	3
1. Introduction.....	4
2. Analyse de la situation.....	6
2.1 Présentation du Centre Antoine Lacassagne.....	6
2.2 Service DSI et contraintes HDS.....	6
2.3 Analyse des besoins.....	7
2.4 Cahier des charges et planning.....	8

3. Conception et réalisation.....	8
3.1 Choix technologiques et argumentation.....	8
Backend : Flask.....	8
Base de données : SQLite via SQLAlchemy.....	9
File de jobs : ThreadPoolExecutor.....	9
Chiffrement des identifiants : Fernet.....	9
3.2 Architecture applicative.....	9
Les jobs planifiés : trois leçons sur la concurrence.....	10
3.3 Sources et synchronisation.....	11
3.4 Fraîcheur, purge et corbeille.....	14
3.5 Interface web et parcours utilisateur.....	15
3.6 API REST.....	17
3.7 Sécurité.....	17
3.8 Tests et qualité.....	18
3.9 Déploiement et exploitation.....	19
4. Bilan et perspectives.....	20
4.1 Résultats obtenus.....	20
4.2 Perspectives et évolutions.....	20
4.3 Accompagnement du CSU lors des interventions.....	20
5. Difficultés rencontrées.....	21
6. Conclusion.....	22
Table des illustrations.....	23
Glossaire.....	23
Bibliographie et sitographie.....	24
Documentation technique.....	24
Sources institutionnelles.....	25

1. Introduction

Lorsqu'un outil informatique du Centre Antoine Lacassagne devient indisponible (panne réseau, mise à jour bloquante, incident serveur), le personnel doit continuer à travailler : les **modes dégradés** répondent à ce besoin par des procédures et formulaires papier ou PDF qui remplacent temporairement les outils habituels.

Ces documents existent dans l'établissement, mais leur gestion reste dispersée : un serveur SFTP pour le bloc opératoire, un partage SMB pour les urgences, un dossier local ailleurs. Aucune vue centralisée ne répondait aux questions simples : « ce document est-il à jour ? », « existe-t-il encore sur le serveur source ? », « quand a-t-il été modifié ? ».

La DSI m'a confié la conception et le développement d'une application web interne pour combler ce vide, avec quatre contraintes simultanées : fédérer des protocoles hétérogènes en un point d'accès unique ; calculer la fraîcheur documentaire sans saturer le réseau ; protéger les identifiants des serveurs en gardant une administration simple ; déployer sans composant lourd.

La problématique du stage peut donc se formuler ainsi : comment centraliser et fiabiliser l'accès à ces documents critiques, tout en conciliant des exigences qui tirent en sens opposés - garantir la fraîcheur documentaire sans saturer le réseau hospitalier, imposer une sécurité et une traçabilité de niveau HDS sans compliquer l'accès en situation de panne, et rester assez simple pour être exploité durablement par la DSI ? L'ensemble de ce rapport est construit autour de cette question, et la conclusion y répond au regard des résultats obtenus.

J'ai conçu et développé Arche en réponse à cette problématique. L'application collecte les PDF depuis les sources déclarées, conserve une copie locale avec déduplication, calcule un état de fraîcheur par document, fournit une interface web de consultation et journalise les opérations. La figure 1 montre le passage d'un accès dispersé sur plusieurs serveurs à un accès centralisé, sécurisé et traçable.

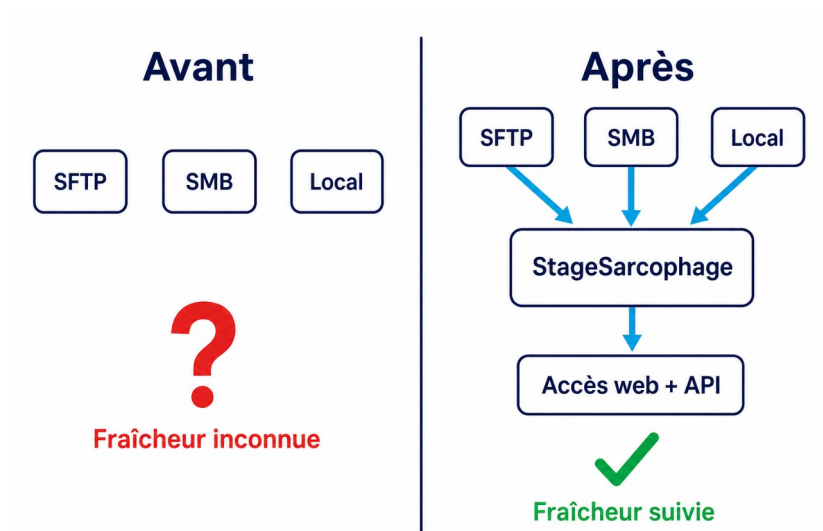


Figure 1 : Situation avant et après Arche : dispersion des sources et accès centralisé.

Le rapport suit un plan problème/solution en trois parties. La première analyse la situation : l'établissement, la DSI, les contraintes HDS et les besoins. La deuxième retrace la conception

dans l'ordre où les problèmes se sont réellement posés : pour chaque sous-système, la difficulté rencontrée, les solutions envisagées et celle que j'ai retenue - y compris mes premières versions imparfaites, souvent les plus instructives. La troisième dresse le bilan et revient sur la seconde mission du stage, l'accompagnement du support aux utilisateurs ; une dernière section rassemble les difficultés transversales, avant que la conclusion ne réponde à la problématique.

2. Analyse de la situation

2.1 Présentation du Centre Antoine Lacassagne

Le Centre Antoine Lacassagne (CAL) est un Centre de Lutte Contre le Cancer (CLCC) niçois, membre du réseau Unicancer ; il accueille chaque année plus de 15 000 patients, emploie environ 800 personnes et couvre les trois missions des CLCC : soins (oncologie, radiothérapie, chirurgie), recherche clinique et formation.

Son système d'information porte des données de santé au sens de la réglementation française - dossiers patients, prescriptions, imagerie médicale - ce qui impose des exigences fortes de sécurité, de traçabilité et de confidentialité. La DSI gère l'infrastructure, les serveurs, les applications métier et la sécurité des systèmes dans ce cadre contraint.

2.2 Service DSI et contraintes HDS

La DSI du CAL intervient sur un périmètre large : gestion des accès, exploitation des serveurs, déploiement d'applications internes, sauvegardes et continuité de service. Le contexte hospitalier impose plusieurs contraintes spécifiques.

Exigences HDS : la certification HDS (Hébergeur de Données de Santé) encadre en France l'hébergement de données médicales : documenter les mesures de sécurité, tracer les accès, formaliser sauvegardes et gestion d'incidents. Elle vise l'hébergement, mais toute application interne doit s'aligner sur les mêmes exigences ; pour Arche, cela s'est traduit par le chiffrement des identifiants, les journaux d'opérations, le contrôle d'accès par rôle et la gestion séparée des secrets.

Continuité d'activité : un établissement de santé doit continuer à fonctionner pendant une panne informatique ; les procédures de mode dégradé (panne du dossier patient, du système de prescription, coupure réseau) reposent sur des documents PDF qui doivent rester accessibles même quand l'outil principal ne l'est plus.

Contraintes de déploiement : la DSI ne souhaitait pas d'infrastructure complexe pour une application interne : SQLite acceptable, pas de serveur de base de données externe, déploiement par conteneur Docker, configuration par variables d'environnement.

2.3 Analyse des besoins

L'analyse des besoins a structuré la demande en trois niveaux, du plus critique au plus opérationnel.

Besoins fonctionnels primaires - le cœur du système, sans lequel rien d'autre n'a de sens :

Déclarer des sources (SFTP, SMB, local) et tester leur connexion ; synchroniser automatiquement les PDF en conservant une copie locale ; suivre la fraîcheur de chaque document (date de modification source, statut) ; permettre consultation, recherche, filtrage et téléchargement depuis une interface web ; journaliser toutes les opérations.

Besoins fonctionnels secondaires - la qualité d'exploitation, qui conditionne l'adoption par la DSI :

Gestion des utilisateurs, des rôles et des permissions (RBAC) ; API REST pour les intégrations futures ; purge automatique avec passage par une corbeille ; notifications email en cas d'erreur ou de document critique.

Contraintes non fonctionnelles - les conditions sine qua non d'un déploiement hospitalier :

Sécurité (chiffrement des identifiants, sessions, en-têtes HTTP, contrôle des chemins) ; déployabilité (Docker + Gunicorn, configuration par variables d'environnement, SQLite) ; maintenabilité (code en couches, tests automatisés, documentation d'exploitation).

2.4 Cahier des charges et planning

Le cahier des charges de la DSI détaille les exigences : sources et connecteurs, synchronisation et déduplication, suivi de fraîcheur, consultation web, sécurité, API, exploitation ; leur état de livraison est dressé en section 4.1.

Le projet a été planifié sur huit semaines : analyse (S1), architecture et modèle de données (S2), connecteurs et sources (S3), synchronisation et fraîcheur (S4), interface web (S5), sécurité et API (S6-S7), tests et documentation (S8).

Le réalisé a suivi ce prévisionnel, à un débordement près : la sécurité, dont la charge croît avec chaque fonctionnalité manipulant fichiers et identifiants. Le décalage a été absorbé en jouant sur le périmètre plutôt que sur la qualité : certaines fonctionnalités avancées ont été documentées plutôt que finalisées.

3. Conception et réalisation

3.1 Choix technologiques et argumentation

Les choix techniques partent du cahier des charges et des contraintes d'exploitation. Chaque composant a été choisi après comparaison avec des alternatives plus lourdes ou moins adaptées au besoin.

Backend : Flask

Le besoin était une application web interne à maintenir par une DSI, pas un service haute disponibilité. Flask correspondait : léger, rendu HTML côté serveur via Jinja natif, déploiement simple derrière Gunicorn. Django aurait apporté plus - administration, ORM, structure imposée - mais précisément ce surplus était un coût sans contrepartie pour ce périmètre. FastAPI aurait été pertinent pour une API principalement asynchrone ; moins pour une application dont l'interface web reste le cas d'usage principal.

Base de données : SQLite via SQLAlchemy

SQLite en mode WAL est embarqué dans Python, ne nécessite aucune installation et se sauvegarde par simple copie de fichier - trois avantages concrets pour un déploiement hospitalier sans DBA dédié. Ses limites d'écritures concurrentes ne sont pas un problème ici : le volume attendu reste faible (quelques synchronisations planifiées, des consultations web, des opérations d'administration). Et si ce volume devait croître, SQLAlchemy laisse ouverte la migration vers PostgreSQL sans réécrire la couche de données.

File de jobs : ThreadPoolExecutor

Un ThreadPoolExecutor interne suffit pour quelques dizaines de synchronisations par jour - sans dépendance supplémentaire. Celery ou RQ offrirait une meilleure persistance des jobs et une reprise propre après redémarrage, mais au prix d'un composant supplémentaire (Redis) et d'un worker séparé à opérer. Ce coût n'était pas justifié pour ce périmètre. La contrepartie - un job perdu si le conteneur s'arrête en pleine synchronisation - est assumée et documentée côté exploitation (section 3.9).

Chiffrement des identifiants : Fernet

Fernet fournit un chiffrement symétrique authentifié avec HMAC-SHA256 intégré. La bibliothèque est maintenue, documentée et évite d'implémenter soi-même AES-CBC, exercice où les erreurs sont faciles sur des points sensibles comme le vecteur d'initialisation ou le code d'authentification. Les sources étant déclarées dynamiquement depuis l'interface, leurs identifiants doivent être stockés en base ; les chiffrer avec une clé conservée hors de la base garantit qu'une copie du fichier SQLite seule ne divulgue aucun identifiant.

3.2 Architecture applicative

Arche est un monolithe Flask - une seule application déployée d'un bloc - organisé en couches distinctes. Ce découpage en couches n'est pas anodin : au départ, les premières routes mélangeaient logique d'accès réseau, règles de stockage et rendu HTML - ce qui rendait les tests impossibles et les bugs difficiles à isoler. Séparer les couches a été la première décision structurelle du projet.

La figure 2 positionne Arche entre les sources hétérogènes, les utilisateurs web et les clients API.

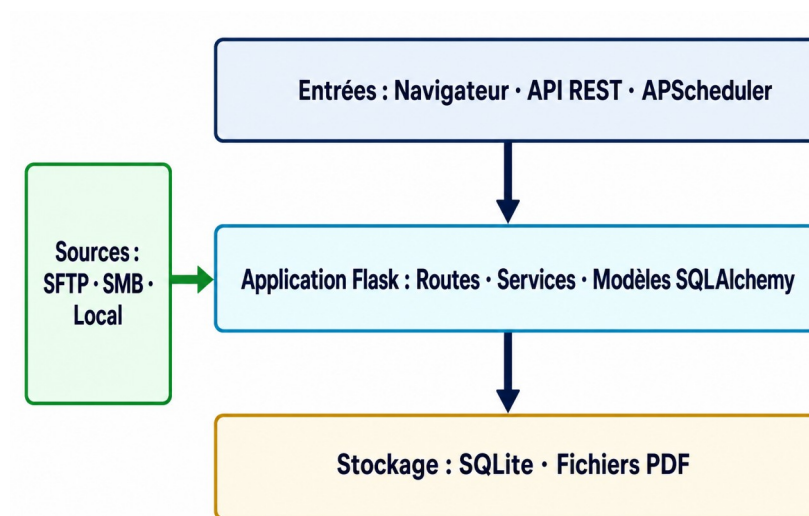


Figure 2 : Architecture applicative d'Arche : couches Flask, services et stockage.

Les routes HTTP (app/routes/) se limitent à l'authentification, à la lecture des paramètres, à la délégation aux services et au rendu. Les services (app/services/) portent la logique métier : synchronisation, purge, jobs de fond, notifications, LDAP, connecteurs. Les modèles SQLAlchemy (app/models/) définissent le schéma de données et les invariants. Les utilitaires (app/utills/) fournissent les fonctions transverses : sécurité des chemins, chiffrement, sanitisation, décorateurs de permissions.

Le modèle de données place Source et Document au centre du flux, liés par une relation un-à-plusieurs. Les autres entités (Journal, BackgroundJob, User, Role, APIToken) couvrent l'exploitation, le suivi des jobs et la sécurité.

Les jobs planifiés : trois leçons sur la concurrence

Cette architecture a été mise à l'épreuve par les jobs planifiés. APScheduler exécute chaque tâche périodique - synchronisation par source, purge à 2 h et nettoyage de la corbeille à 3 h (deux mécanismes détaillés en section 3.4), rapport de conformité à 8 h (un état quotidien de la fraîcheur du parc documentaire) - dans un thread séparé du processus Flask. Cette cohabitation entre threads de fond et requêtes web m'a confronté à trois problèmes successifs.

Le premier est apparu dès le premier essai : `RuntimeError: Working outside of application context`. Le même code fonctionnait depuis une route web et plantait appelé par le scheduler : la configuration et la session de base de données sont attachées à un *contexte d'application* que les requêtes web reçoivent automatiquement - mais pas les threads créés par APScheduler. La correction consiste à envelopper le corps de chaque job dans `with app.app_context():` (`app/scheduler/tasks.py`). Une ligne, mais elle m'a obligé à comprendre comment Flask isole réellement ses requêtes.

Le deuxième problème était un chevauchement : avec une fréquence de test courte et une source lente, le job suivant démarrait avant la fin du précédent - deux threads écrivaient sur les mêmes lignes en base. APScheduler fournit exactement les garde-fous nécessaires : `max_instances=1` interdit deux exécutions simultanées du même job, et `misfire_grace_time=60` autorise un déclenchement en retard plutôt qu'une exécution de rattrapage en rafale. Une garde au démarrage évite enfin le double scheduler créé par le rechargeur automatique de Flask en développement.

Le troisième touchait SQLite : synchronisation planifiée active et interface ouverte en parallèle, certaines requêtes échouaient avec `sqlite3.OperationalError: database is locked` : le thread du scheduler écrivait pendant qu'une requête web lisait. Plutôt que de remettre en cause le choix de SQLite (section 3.1), j'ai activé le mode WAL à l'ouverture de chaque connexion (`PRAGMA journal_mode=WAL` dans `app/__init__.py`). Dans ce mode, l'écriture se fait dans un journal séparé et ne bloque plus les lectures - exactement notre profil d'usage : écritures occasionnelles, lectures fréquentes.

Ces fondations posées - couches séparées, jobs maîtrisés, base fiabilisée -, le cœur fonctionnel restait à construire : la collecte des documents elle-même.

3.3 Sources et synchronisation

Une source décrit l'origine des PDF : protocole (sftp, smb, local), adresse réseau, chemin distant, identifiants chiffrés, filtres, seuils de fraîcheur et durée de rétention. Dans le code, les identifiants sont manipulés via les propriétés Python `login` et `mot_de_passe` de `Source` ; les colonnes physiques en base stockent les valeurs chiffrées par Fernet.

Le service `app/services/sync_service.py` applique la même logique à tous les protocoles : seul le connecteur (inventaire et téléchargement) change. Cette partie est celle qui a connu le plus d'itérations pendant le stage, et c'est en la racontant dans l'ordre qu'on comprend le mieux sa forme finale.

Première version : correcte mais inutilisable. Ma première implémentation téléchargeait chaque fichier distant, calculait son hash SHA-256 et ne remplaçait la copie locale qu'en cas de changement. Fonctionnellement correct ; mais chaque synchronisation re-téléchargeait l'intégralité de la source, car il faut posséder un fichier pour le hacher. Face à l'objectif du cahier des charges (500 PDF en moins de 5 minutes), cette approche ne tenait pas.

Deuxième version : déduplication à deux niveaux. La solution exploite les métadonnées que SFTP et SMB fournissent gratuitement à l'inventaire - taille et date de modification : si les deux correspondent à la base, le fichier est ignoré d'emblée. Le hash ne sert plus qu'en confirmation après téléchargement, contre les fichiers re-datés sans modification. Le coût réseau devient proportionnel à ce qui a *changé*, et non à la taille totale de la source.

Le bug des deux secondes. Ce pré-filtre a fait apparaître un bug déroutant : certains fichiers - toujours les mêmes - étaient re-téléchargés sans avoir changé. En journalisant les comparaisons, j'ai constaté des écarts d'une seconde : plusieurs systèmes de fichiers et serveurs SMB arrondissent les dates de modification à deux secondes près, et l'arrondi peut différer entre le serveur et la base. La correction tient en une condition : tolérer un écart inférieur à deux secondes.

```
# sync_service.py - pré-filtre avant téléchargement (simplifié)
if doc and doc.taille_octets == f_distant.taille:
    doc_mtime = _as_utc(doc.date_modification_source)
    if doc_mtime:
        ecart = abs((doc_mtime - f_distant.date_modification).total_seconds())
        if ecart < 2: # tolérance : granularité des dates SMB/FAT
            return "ignore" # métadonnées identiques : pas de téléchargement
```

Les fuseaux horaires. Le connecteur SFTP a ensuite levé une erreur nouvelle pour moi : `TypeError: can't subtract offset-naive and offset-aware datetimes`. Paramiko renvoie des dates « naïves » (sans fuseau), l'application travaille en UTC « aware » ; Python refuse de les soustraire, à raison. Règle adoptée pour tout le projet : tout est stocké et comparé en UTC, la conversion locale n'intervient qu'à l'affichage ; un petit utilitaire normalise les dates entrantes :

```
def _as_utc(dt: datetime | None) -> datetime | None:
    if dt is None:
        return None
    return dt if dt.tzinfo else dt.replace(tzinfo=timezone.utc)
```

Écriture contrôlée. Le téléchargement passe par un fichier temporaire, renommé seulement une fois complet et supprimé en cas d'exception : sans cela, une coupure réseau laisserait un PDF tronqué servi tel quel à l'utilisateur - une défaillance silencieuse interdite pour un outil de continuité d'activité. Le calcul du hash lit par blocs de 64 Ko : certains PDF sont des scans de dizaines de mégaoctets, et `sha256(f.read())` aurait chargé chacun intégralement en mémoire.

La figure 3 résume la chaîne complète : inventaire, sécurisation du nom, les deux niveaux de déduplication, écriture contrôlée et journalisation de chaque issue (copié, ignoré, erreur).

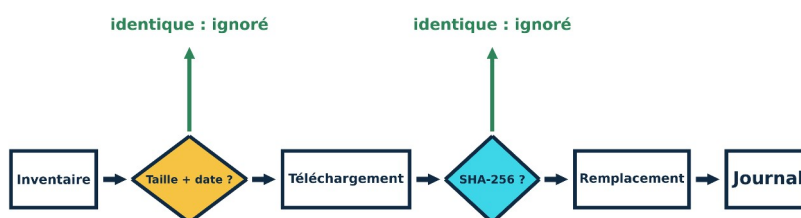


Figure 3 : Flux de synchronisation d'un document PDF : déduplication à deux niveaux et écriture contrôlée.

Les noms de fichiers, une entrée comme une autre. Le point le plus sensible concerne les noms de fichiers distants : un fichier nommé `../secret.pdf` sur un serveur compromis doit être refusé avant tout stockage local. Ma première protection s'appuyait sur `os.path.basename()` pour ne conserver que le nom - jusqu'à ce qu'un test m'apprenne que sous Linux, `basename` ne coupe que sur les `/` : un nom contenant des antislashes Windows (`..\..\fichier.pdf`), tout à fait plausible venant d'un serveur SMB, traversait le filtre intact. La fonction `nom_fichier_sur()` (`app/utls/files.py`) normalise donc d'abord les `\` en `/` avant d'extraire le nom, et le confinement par `realpath + commonpath` vérifie en seconde ligne de défense que le chemin final reste bien sous le répertoire de stockage. Cette double vérification s'applique partout où un chemin est manipulé.

La synchronisation répond à la question « comment collecter efficacement ». Restait la question symétrique, tout aussi importante pour la DSI : que deviennent les documents qui vieillissent ?

3.4 Fraîcheur, purge et corbeille

Chaque document porte un statut calculé depuis son âge et les seuils de sa source : ok (document récent), avertissement (à surveiller), critique (action requise), puis purge (rétention dépassée). Cette gradation laisse le temps de réagir, et les seuils par source permettent des politiques adaptées à la criticité des procédures. La purge répond à un risque métier réel : appliquer une procédure obsolète peut être plus dangereux que de constater son absence ; écarter un document que sa source ne maintient plus incite le service responsable à en republier une version à jour.

La suppression elle-même n'est jamais immédiate. Quand un document dépasse son seuil de rétention, il est déplacé vers le dossier _corbeille et son statut passe à purge. Un nettoyage définitif intervient ensuite après une durée configurable (paramètre CORBEILLE_RETENTION_JOURS). Ce fonctionnement en deux temps limite le risque d'erreur irréversible en cas de mauvais paramétrage. La figure 4 illustre l'ensemble de ce cycle de vie et les seuils de transition.

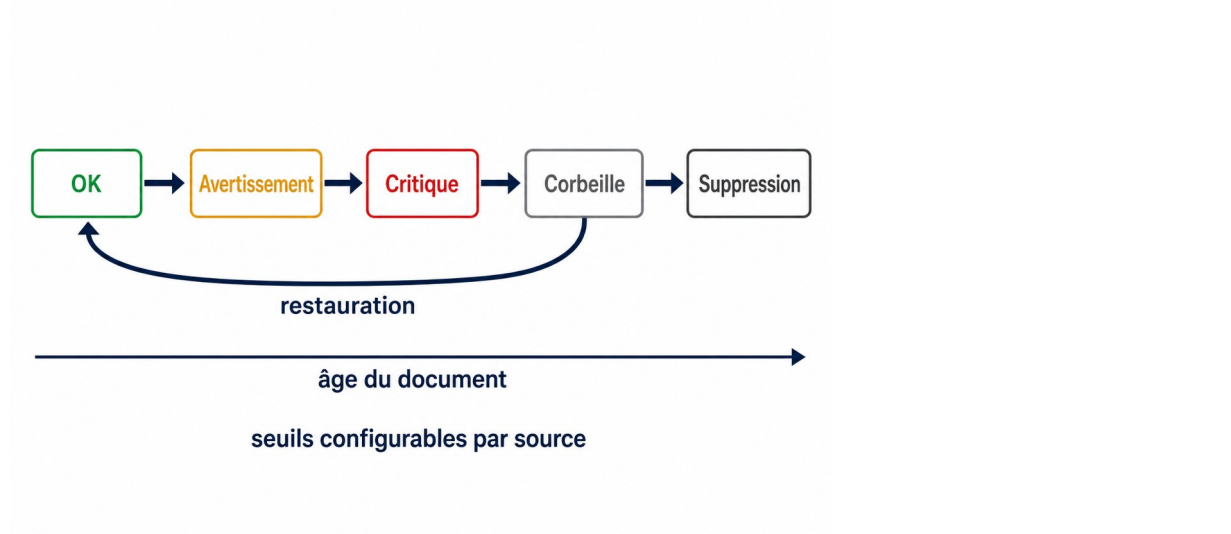


Figure 4 : Cycle de vie d'un document PDF : états de fraîcheur, transitions et chronologie de purge.

Ces mécanismes tournent en arrière-plan ; pour l'utilisateur, ils se résument à un badge de couleur sur chaque document. C'est l'interface web qui rend ce travail visible.

3.5 Interface web et parcours utilisateur

L'interface est rendue côté serveur avec Jinja2 et Bootstrap 5. Ce choix a une raison pratique : un développeur Python de la DSI doit pouvoir maintenir l'interface sans maîtriser un framework JavaScript. Les écrans de gestion interne n'exigent pas de réactivité temps réel. L'interface couvre la connexion, le tableau de bord, la gestion des sources, la synchronisation, la consultation des documents, les journaux et l'administration. Les figures 5 à 8 montrent ces écrans tels qu'ils se présentent en exploitation.

Le tableau de bord synthétise l'état du parc : volume de documents suivis, part à jour, documents en avertissement ou critiques, état de chaque source et derniers documents à traiter (figure 5).

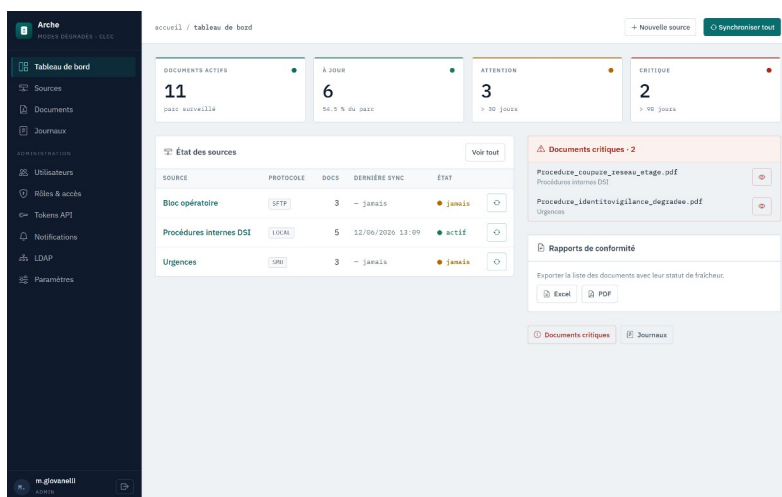


Figure 5 : Tableau de bord d'Arche : état du parc documentaire et des sources.

La consultation s'appuie sur la recherche et les filtres (source, statut, période) ; un badge de couleur matérialise la fraîcheur de chaque document (figure 6). Le viewer intégré affiche le PDF sans téléchargement, accompagné de ses métadonnées de collecte : source, hash SHA-256, dates et statut (figure 7).

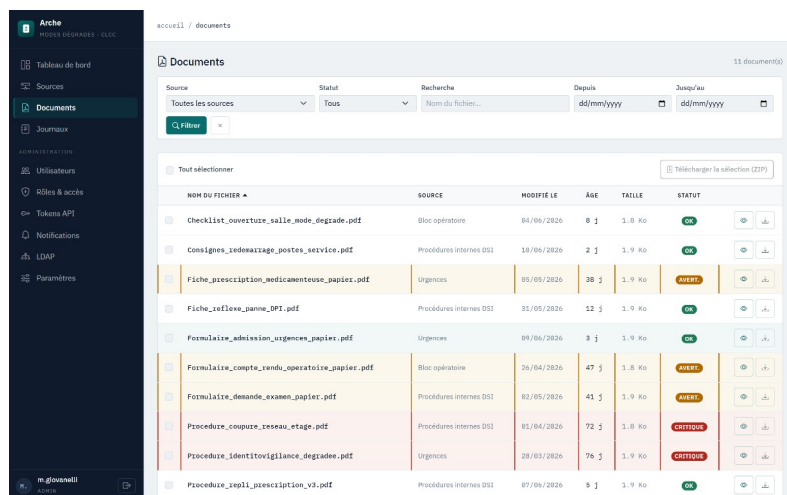


Figure 6 : Consultation des documents : filtres, recherche et statuts de fraîcheur.

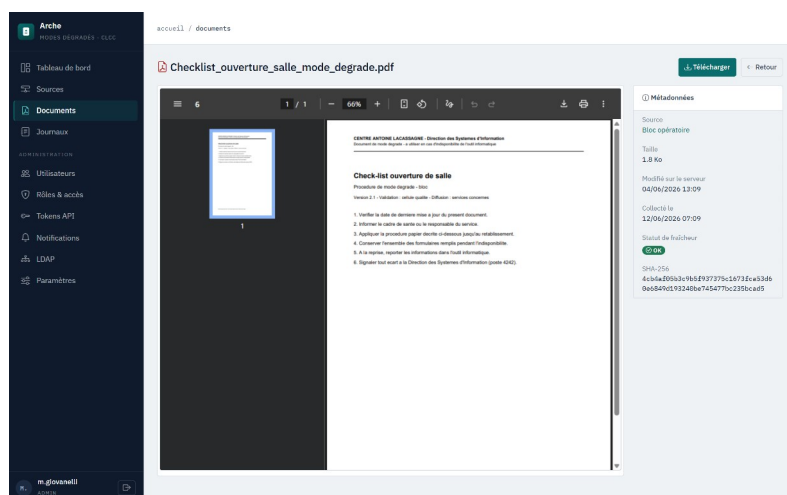


Figure 7 : Viewer PDF intégré : lecture du document et métadonnées de collecte.

La gestion des sources regroupe la déclaration des serveurs, le test de connexion et le déclenchement manuel d'une synchronisation (figure 8).

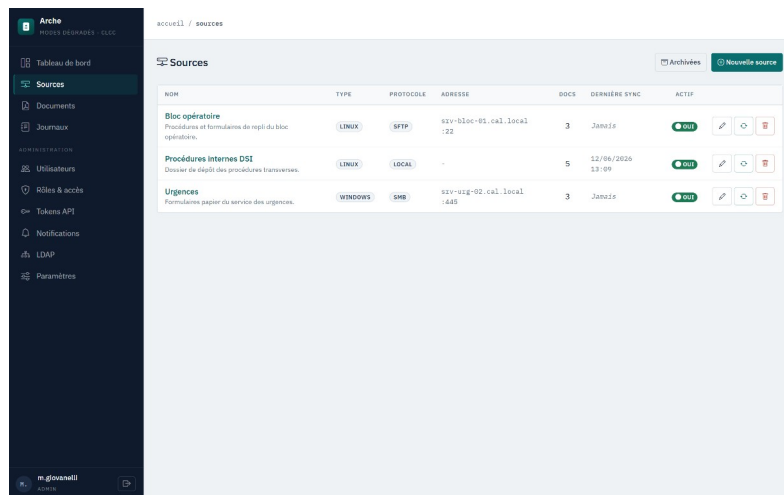


Figure 8 : Gestion des sources : protocoles, état et synchronisation.

L'export ZIP d'une sélection est plafonné à 100 documents et 500 Mo, pour éviter une réponse HTTP qui saturerait la mémoire du serveur ou la connexion de l'utilisateur.

L'interface répond ainsi à l'usage humain. Pour les usages programmatiques - supervision par les outils de la DSI, scripts d'exploitation -, l'application expose une API REST.

3.6 API REST

L'API v1 (app/routes/api.py, documentée via OpenAPI) expose un healthcheck public, des statistiques agrégées, la liste et le détail des sources, le déclenchement d'une synchronisation, le statut d'un job de fond, la liste et le détail des documents, ainsi que le téléchargement d'un document.

L'API n'est pas un CRUD complet - choix délibéré : créer ou supprimer une source est une opération rare, mieux servie par les confirmations visuelles de l'interface ; l'API se concentre sur la consultation et le suivi, les vrais besoins d'un monitoring externe. Les décorateurs dans app/utils/decorators.py et les tests dans tests/test_api_permissions.py garantissent que les deux modes d'accès (session web et token Bearer) appliquent les mêmes règles de permissions, sans divergence silencieuse.

3.7 Sécurité

La sécurité d'Arche repose sur des contrôles complémentaires - aucun ne suffit seul -, regroupés ici par type de menace.

Contre l'usurpation d'identité : Flask-Login gère les sessions côté serveur, avec mots de passe hachés en bcrypt (coût 12) et décorateurs `@require_permission` sur les routes sensibles. L'authentification peut aussi être déléguée à l'annuaire LDAP de l'établissement. Les tokens Bearer sont stockés hachés en base, révocables immédiatement, et peuvent porter des permissions restreintes. Rôles comme tokens limitent les droits au nécessaire : « Opérateur », par exemple, ne peut pas supprimer une source.

Contre les attaques par le navigateur : Flask-WTF pose des jetons CSRF sur tous les formulaires qui modifient des données, ce qui empêche un attaquant de déclencher une action en s'appuyant sur la session ouverte d'un utilisateur. Les en-têtes HTTP (X-Content-Type-Options: nosniff, X-Frame-Options: SAMEORIGIN, CSP avec nonce dynamique sur les scripts) réduisent les vecteurs XSS et clickjacking.

La mise en place de la CSP a été instructive : en l'activant, l'interface a cessé de fonctionner - les scripts inline des templates étaient bloqués comme l'aurait été un script injecté, car la CSP exige que le code légitime soit identifiable. Chaque réponse génère donc un nonce aléatoire, placé dans l'en-tête CSP et dans les balises `<script>` ; un script dépourvu du nonce de la requête en cours est refusé par le navigateur.

Contre les attaques par le système de fichiers : les identifiants des sources sont chiffrés avec Fernet avant stockage - la clé (ENCRYPTION_KEY) reste séparée de la clé de session et protégée par des permissions restreintes (chmod 600). Le confinement des chemins dans `app/utils/files.py` utilise `os.path.realpath` et `os.path.commonpath` pour garantir que tout chemin demandé reste sous le répertoire autorisé, y compris face aux liens symboliques ou aux chemins absolus injectés.

Contre les injections de formules : les exports CSV/XLSX neutralisent les valeurs commençant par `=`, `+`, `-` ou `@`. Les notifications et rapports HTML échappent systématiquement les contenus insérés.

3.8 Tests et qualité

La stratégie de test a privilégié la rapidité et la reproductibilité. Les tests utilisent SQLite en mémoire et un stockage temporaire ; le mode `JOBS_RUN_INLINE=true` exécute les jobs de fond de façon synchrone. Ce mode est né d'un problème concret : mes premiers tests

échouaient par intermittence, selon que le thread du job avait fini ou non au moment de l'assertion ; plutôt que des attentes arbitraires (sleep), l'exécution en ligne des jobs rend les assertions déterministes. Résultat : une suite qui tourne en quelques secondes en CI, couvrant modèles, synchronisation, purge, sources, permissions web et API, sécurité, connecteurs, exports, notifications et templates.

La contrepartie est connue : ces tests ne prouvent pas la compatibilité avec les comportements réels de serveurs SFTP, SMB, LDAP ou SMTP. Une recette sur infrastructure réelle reste indispensable avant tout déploiement.

La commande `make check` orchestre l'ensemble : Ruff (linter Python), pytest avec couverture, Bandit (analyse statique de sécurité), pip-audit (vulnérabilités des dépendances), un scan de secrets suivis par Git et une vérification des permissions du fichier `.env`.

3.9 Déploiement et exploitation

Le dépôt contient un `Dockerfile`, un `docker-compose.yml`, un `entrypoint.sh` et les commandes Flask nécessaires à l'initialisation de la base.

Variables minimales requises en production :

```
SECRET_KEY=<valeur aléatoire 32 octets>
ENCRYPTION_KEY=<clé Fernet générée>
STORAGE_DIR=/chemin/vers/stockage
FLASK_ENV=production
```

Le déploiement cible passe par Docker (le lancement local hors conteneur est décrit dans le README) :

```
docker compose build && docker compose up -d
docker compose exec web flask init-db
docker compose exec web flask create-admin --username admin --password <mot_de_passe>
```

Un redémarrage pendant une synchronisation peut laisser un job inachevé, à relancer manuellement : limite connue, documentée pour l'exploitant (voir section 3.1).

4. Bilan et perspectives

4.1 Résultats obtenus

Arche couvre le périmètre principal du cahier des charges : une application web interne, conteneurisable, qui collecte des PDF depuis des sources SFTP, SMB et locales, calcule leur fraîcheur, permet leur consultation et trace les opérations.

Sur les exigences du cahier des charges, l'essentiel est livré et validé par les tests : gestion des sources, synchronisation avec déduplication SHA-256, consultation (interface web, viewer PDF, export ZIP), sécurité (CSRF, CSP, Fernet, RBAC, confinement de chemins) et API REST. Deux points restent partiels : les réglages fins de la purge n'ont pas tous été démontrés en recette ; l'API ne couvre pas le CRUD des sources. Trois points restent à valider sur infrastructure réelle : LDAP, SMTP et le benchmark à 500 PDF. Le projet est documenté pour une reprise par la DSI sans dépendre de ma présence.

4.2 Perspectives et évolutions

Priorité haute : valider les connecteurs sur infrastructure réelle (SFTP, SMB, LDAP, SMTP) et mesurer le benchmark 500 PDF en moins de 5 minutes - prérequis à toute mise en service.

Priorité moyenne : ajouter le CRUD des sources dans l'API REST ; automatiser la sauvegarde de la base, du stockage PDF et du fichier .env.

Priorité basse : envisager une migration PostgreSQL si les accès concurrents l'exigent ; faire réaliser un audit de sécurité externe une fois le périmètre de production figé.

4.3 Accompagnement du CSU lors des interventions

Le bilan du stage ne se limite pas à l'application : en plus du développement d'Arche, j'ai accompagné la Cellule de Support aux Utilisateurs (CSU) lors d'interventions sur site (bloc opératoire, urgences, unités d'hospitalisation) et à distance - dépannages de postes, droits d'accès, incidents d'impression, connectivité réseau. Cette seconde mission m'a donné une vision directe du terrain que l'application devait servir : postes partagés, politiques de sécurité strictes, applications métier contrôlées par profil et par service.

Le contexte hospitalier se distingue surtout par la continuité de service : une panne dans un service de soins perturbe directement la prise en charge des patients. Lors d'une intervention, j'ai vu une unité revenir aux formulaires papier à cause d'une indisponibilité des outils de prescription - exactement le scénario qu'Arche sécurise. Ces accompagnements ont aussi influencé la conception : le personnel hospitalier a des niveaux d'aisance informatique très variables, et une application utilisée en contexte de panne doit afficher des messages d'état compréhensibles et éviter les parcours inutiles.

5. Difficultés rencontrées

Les difficultés techniques ponctuelles ont été racontées dans la partie 3, au plus près des choix qu'elles ont provoqués. Cette section revient sur les difficultés transversales, celles qui ont structuré le déroulement du stage.

Mener deux missions de front. Les accompagnements du CSU, par nature imprévisibles, morcelaient le temps de développement. Il a fallu protéger des plages continues - une fonctionnalité comme la synchronisation supporte mal le morcellement - tout en restant disponible pour les interventions ; d'où les arbitrages de périmètre évoqués en section 2.4.

Changer de posture sur la sécurité. La difficulté la plus formatrice a été de changer de question : ne plus me demander « est-ce que ça marche ? » mais « comment cela peut-il être détourné ? ». Chaque fonctionnalité ajoutée rouvrait la question pour un périmètre nouveau - noms de fichiers distants, exports tableur, chemins, tokens - et c'est ce qui explique le débordement de la phase sécurité sur le planning prévisionnel.

Garantir les mêmes règles sur deux chemins d'accès. L'interface web (sessions, CSRF) et l'API (tokens Bearer) doivent appliquer exactement les mêmes permissions métier. J'ai centralisé les vérifications dans des décorateurs communs et verrouillé cette propriété par des tests dédiés : si les deux chemins divergent un jour, un test échouera.

Tester sans l'infrastructure cible. SFTP, SMB, LDAP et SMTP ont été simulés avec unittest.mock, ce qui rend reproductibles les succès, les erreurs réseau et les entrées dangereuses. Ces tests valident ma logique, pas la compatibilité réelle avec les serveurs de la DSI - d'où la recette en tête des perspectives.

Estimer son propre travail. Le planning s'est avéré juste dans ses grandes lignes mais optimiste sur les finitions : un connecteur « qui marche » et un connecteur testé, sécurisé et journalisé ne représentent pas le même volume de travail. J'en retiens que le dernier quart d'une fonctionnalité coûte aussi cher que les trois premiers.

6. Conclusion

Arche centralise ce qui était dispersé : les PDF de modes dégradés, auparavant éparpillés sur des serveurs hétérogènes, sont maintenant collectés, dédupliqués et accessibles depuis un point d'accès unique. L'application présente les trois attributs non négociables en contexte HDS : elle est sécurisée (Fernet, CSRF, confinement des chemins), traçable (journaux de toutes les opérations) et fiable (copies locales disponibles même si la source est inaccessible).

La réponse à la problématique tient en trois compromis assumés : la déduplication à deux niveaux garantit la fraîcheur documentaire sans saturer le réseau ; le double modèle d'accès session/token, appuyé sur les journaux, impose la sécurité et la traçabilité attendues en contexte HDS sans compliquer la consultation ; et la pile technique volontairement réduite reste exploitable durablement par la DSI.

Les choix techniques - Flask, SQLite, APScheduler, Fernet, ThreadPoolExecutor, Docker - ont été guidés par le cahier des charges : livrer une base maintenable plutôt que multiplier les composants, chaque choix étant comparé aux alternatives et ses limites documentées.

Sur le plan personnel, ce stage m'a davantage appris par ses problèmes que par ce qui a fonctionné du premier coup : un `TypeError` sur des fuseaux horaires, une base verrouillée par un thread ou un job exécuté en double obligent à comprendre ce que les frameworks font réellement sous la surface. C'est cette compréhension - plus que la liste des fonctionnalités livrées - qui me semble constituer l'acquis principal de ces huit semaines.

La prochaine étape est une recette par la DSI sur l'infrastructure réelle du Centre Antoine Lacassagne. Elle permettra de valider les connecteurs, de mesurer les performances sur des fichiers représentatifs et d'ajuster les paramètres de rétention avant une mise en service opérationnelle - ce que ce stage n'a pas pu faire, mais a préparé.

Table des illustrations

Figure 1 : Situation avant et après Arche : dispersion des sources et accès centralisé section 1

Figure 2 : Architecture applicative d'Arche : couches Flask, services et stockage section 3.2

Figure 3 : Flux de synchronisation d'un document PDF : déduplication à deux niveaux et écriture contrôlée section 3.3

Figure 4 : Cycle de vie d'un document PDF : états de fraîcheur, transitions et chronologie de purge section 3.4

Figure 5 : Tableau de bord d'Arche : état du parc documentaire et des sources section 3.5

Figure 6 : Consultation des documents : filtres, recherche et statuts de fraîcheur section 3.5

Figure 7 : Viewer PDF intégré : lecture du document et métadonnées de collecte section 3.5

Figure 8 : Gestion des sources : protocoles, état et synchronisation section 3.5

Glossaire

API REST : interface de programmation exposée via HTTP, permettant à d'autres logiciels d'interroger l'application.

CI : Continuous Integration : exécution automatique des tests à chaque modification du code.

CRUD : Create, Read, Update, Delete : les quatre opérations élémentaires sur une donnée.

CSP : Content Security Policy : en-tête HTTP limitant les ressources chargées par le navigateur, ce qui réduit la surface d'attaque XSS.

CSRF : Cross-Site Request Forgery : attaque forçant un navigateur authentifié à envoyer une requête non voulue ; contrée par un jeton sur chaque formulaire.

DBA : Database Administrator : administrateur de base de données.

Docker : technologie de conteneurs : l'application et ses dépendances sont empaquetées pour un déploiement reproductible.

Fernet : schéma de chiffrement symétrique authentifié (AES-128-CBC + HMAC-SHA256), utilisé pour chiffrer les identifiants des sources en base.

Gunicorn : serveur d'application Python utilisé en production pour exécuter l'application web.

HDS : Hébergeur de Données de Santé : certification française encadrant l'hébergement de données médicales.

LDAP : Lightweight Directory Access Protocol : annuaire d'entreprise ; utilisé ici pour déléguer l'authentification.

Mode dégradé : procédure ou document utilisé lorsque le fonctionnement normal d'un outil métier n'est plus disponible.

Nonce : valeur aléatoire à usage unique, exigée par la CSP pour autoriser un script.

ORM : Object-Relational Mapping : fait correspondre les objets du code aux tables de la base (ici SQLAlchemy).

RBAC : Role-Based Access Control : les droits sont attribués à des rôles, eux-mêmes assignés aux utilisateurs.

Recette : tests menés par le destinataire (ici la DSI) pour valider la conformité avant mise en service.

SFTP : SSH File Transfer Protocol : transfert de fichiers sécurisé via SSH, utilisé pour les sources Linux.

SMB/CIFS : Server Message Block / Common Internet File System : protocole d'accès aux partages de fichiers Windows.

SMTP : Simple Mail Transfer Protocol : protocole d'envoi des emails (notifications).

SQLite WAL : Write-Ahead Logging : mode d'écriture de SQLite améliorant les performances en lecture concurrente.

Token Bearer : jeton secret transmis dans l'en-tête Authorization ; authentifie les appels à l'API.

XSS : Cross-Site Scripting : injection d'un script malveillant dans une page consultée par d'autres utilisateurs.

Bibliographie et sitographie

Documentation technique

- Jordan Borean et al., *smbprotocol Documentation*, dépôt GitHub du projet smbprotocol, (consulté le 12 juin 2026), disponible sur <https://github.com/jborean93/smbprotocol>

- Jeff Forcier et al., *Paramiko Documentation*, site officiel de Paramiko, (consulté le 12 juin 2026), disponible sur <https://www.paramiko.org/>
- Alex Grönholm, *APScheduler Documentation*, documentation en ligne d'APScheduler, (consulté le 12 juin 2026), disponible sur <https://apscheduler.readthedocs.io/>
- Pallets, *Flask Documentation v3.x*, site officiel de Flask, (consulté le 12 juin 2026), disponible sur <https://flask.palletsprojects.com/>
- Python Cryptographic Authority, *cryptography : Fernet*, documentation en ligne de cryptography, (consulté le 12 juin 2026), disponible sur <https://cryptography.io/en/latest/fernet/>
- SQLAlchemy Authors, *SQLAlchemy 2.0 Documentation*, site officiel de SQLAlchemy, (consulté le 12 juin 2026), disponible sur <https://docs.sqlalchemy.org/>

Sources institutionnelles

- ANS (Agence du Numérique en Santé), *Certification HDS*, portail e-santé du ministère de la Santé, (consulté le 12 juin 2026), disponible sur <https://esante.gouv.fr/labels-certifications/hds>
- ANSSI, *Guide des bonnes pratiques de l'informatique*, site officiel de l'ANSSI, (consulté le 12 juin 2026), disponible sur <https://www.ssi.gouv.fr/guide/bonnes-pratiques-de-linformatique/>
- Centre Antoine Lacassagne, *Site institutionnel du Centre Antoine Lacassagne*, site officiel de l'établissement, (consulté le 12 juin 2026), disponible sur <https://www.centreantoinelacassagne.org/>
- Unicancer, *Fiche établissement Centre Antoine Lacassagne*, site officiel d'Unicancer, (consulté le 12 juin 2026), disponible sur <https://www.unicancer.fr/fr/clcc/centre-antoine-lacassagne/>